DIGITAL TWIN

A Exercise Report

(Value Added Course in TANCAM-PMIST)

Submitted By

M MOHANA SUNDHARAM (1240120131205)

B.TECH-Electronics and communication engineering

A-Section

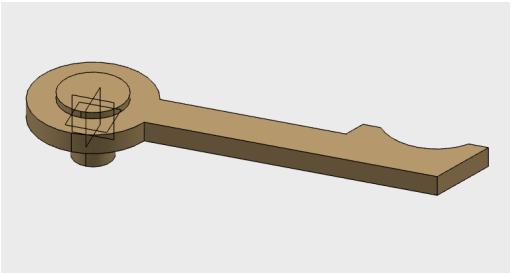
In partial fulfilment for the award of the value added course cerficate on

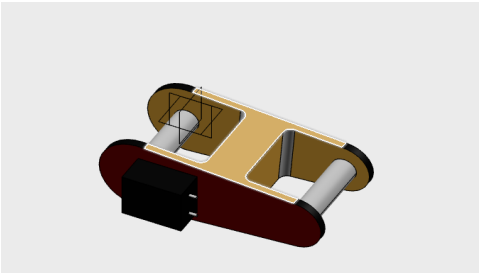
Digital Twin using 3DEXPERIENCE software

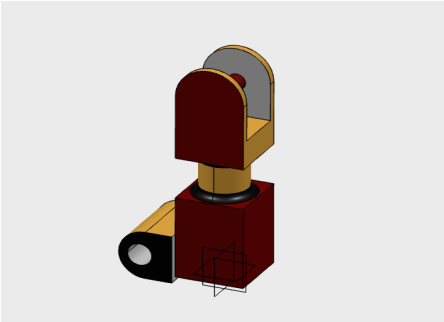
INTRODUCTION:

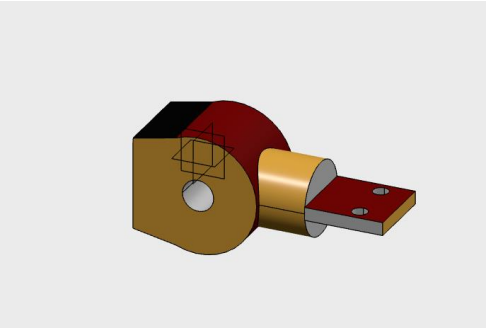
Digital Twin is a virtual representation of a physical product or system. It enables real-time monitoring, simulation, and optimization of performance. 3D Experience software provides a collaborative platform for creating and managing these models. With tools for design, engineering, and lifecycle management, it supports innovation and efficiency. Together, Digital Twin and 3D Experience empower industries to achieve smarter, connected product development.

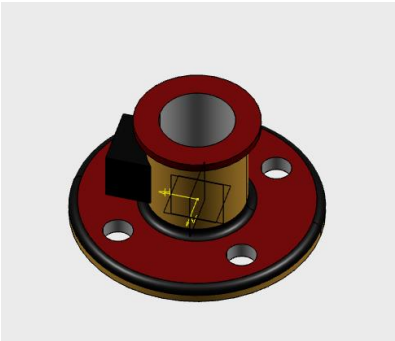
Task 1: Part Design











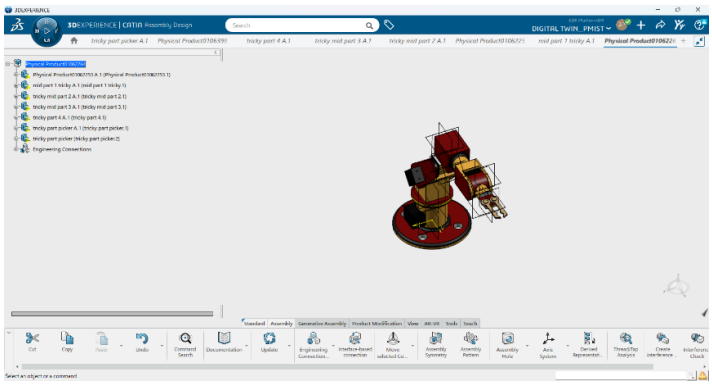
- Part Design begins with creating a sketch on a selected reference plane.
- The sketch is dimensioned and constrained to capture design intent.
- Extrusion, pad, or revolve features are applied to generate the 3D solid.

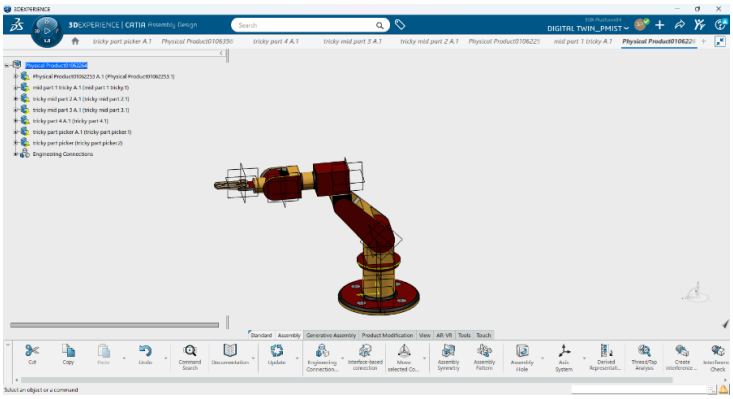
- Additional operations such as holes, pockets, fillets, and chamfers refine the geometry.

- The completed part is validated with material assignment and integrated into assemblies.

Task 2: parts Assembly

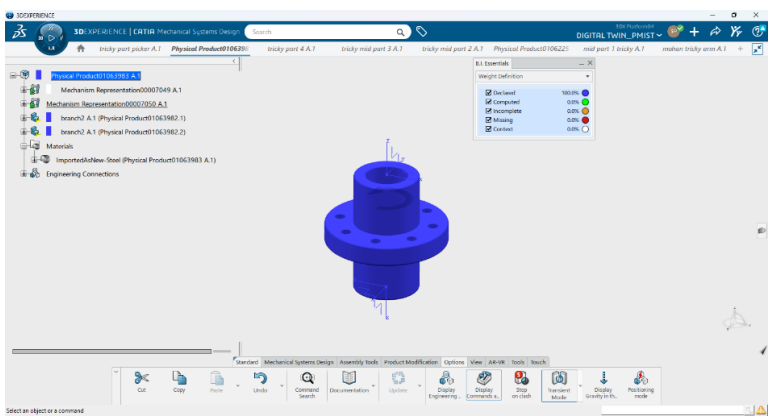
- An assembly design was created in 3D Experience using two components.
- A **coincident constraint** was applied to align the components precisely.
- A **contact constraint** was defined to establish realistic surface interaction.
- The resulting assembly ensures accurate positioning and mechanical behavior.

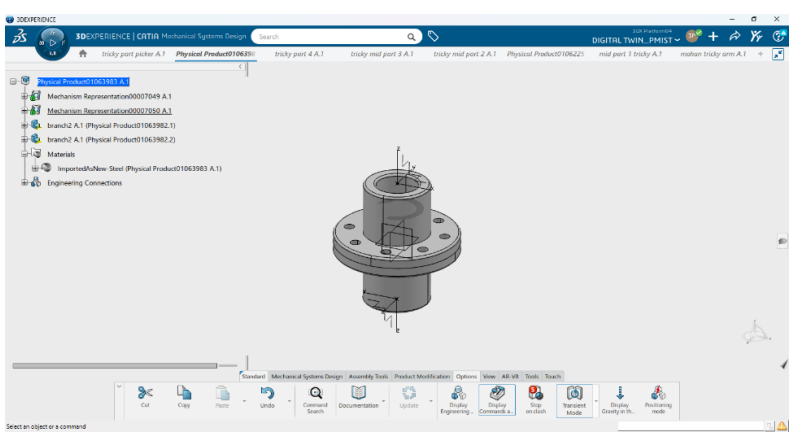




Task 3: Flange design and kinematics Simulation

- The flange design process begins with selecting the appropriate reference plane for sketching.
- A circular or profile sketch is created to define the flange geometry.
- The sketch is extruded to the required thickness, forming the base flange body.
- Additional features such as bolt holes, fillets, or chamfers are incorporated to meet design standards.
- Dimensional constraints are applied to ensure accuracy and consistency in the flange model.
- Material properties are assigned to simulate realistic behavior under load conditions.
- A kinematic simulation is set up to analyze the flange's rotational movement.
- The flange component is rotated through 180 degrees to validate clearance and assembly fit.
- Contact constraints are applied during simulation to check for interference or collision.
- The results confirm proper alignment, smooth rotation, and mechanical feasibility of the design.



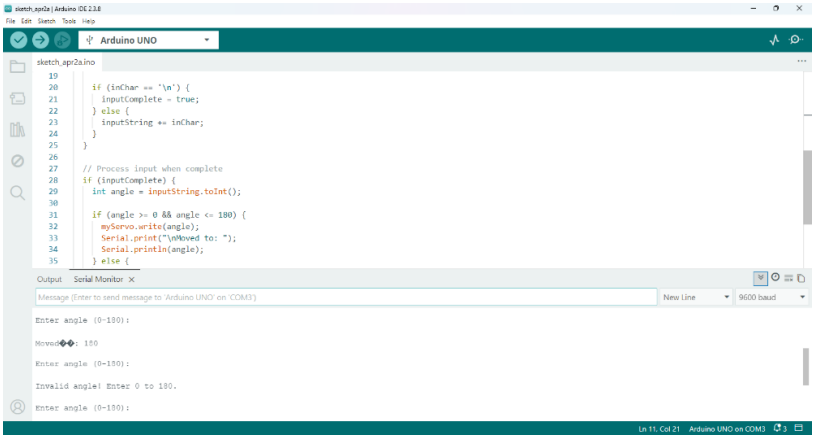


Task 4: Arduino interface with servo

- The servo motor is connected to the Arduino board using a digital PWM pin.
- The control (signal) wire of the servo is attached to **Pin 9** of the Arduino Uno.
- The power (VCC) wire of the servo is connected to the **5V pin** of the Arduino.
- The ground (GND) wire of the servo is connected to the **GND pin** of the Arduino.
- The Servo library is initialized in the Arduino IDE to generate control pulses.
- A program is uploaded to rotate the servo shaft through the desired angle.
- For simulation, a command is issued to rotate the servo by **180 degrees**.

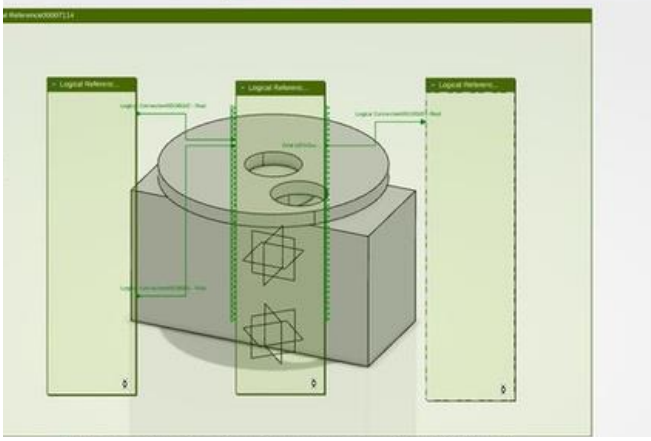
Code:

```
#include <Servo.h>
Servo myServo;
void setup() {
  myServo.attach(3);       // Servo signal wire on pin 3
  Serial.begin(9600);
  Serial.println("Enter angle (0-180):");
}
void loop() {
  if (Serial.available() > 0) {
    int angle = Serial.parseInt();
    if (angle >= 0 && angle <= 180) {
      myServo.write(angle);
      Serial.print("Moved to: ");
      Serial.println(angle);
    } else {
      Serial.println("Invalid! Enter 0 to 180.");
    }    while (Serial.available()) Serial.read();}}
```


sketch_apr2a | Arduino IDE 2.3.6
File Edit Sketch Tools Help
Arduino UNO
sketch_apr2a.ino
if (inChar == '\n') {
inputComplete = true;
} else {
inputString += inChar;
}
}
// Process input when complete
if (inputComplete) {
int angle = inputString.toInt();
if (angle >= 0 && angle <= 180) {
myServo.write(angle);
Serial.print("\nMoved to: ");
Serial.println(angle);
} else {
Output Serial Monitor
Message (Enter to send message to 'Arduino UNO' on 'COM3')
New Line
9600 baud
Enter angle (0-180):
Moved: 180
Enter angle (0-180):
Invalid angle! Enter 0 to 180.
Enter angle (0-180):
Ln 11, Col 21 Arduino UNO on COM3

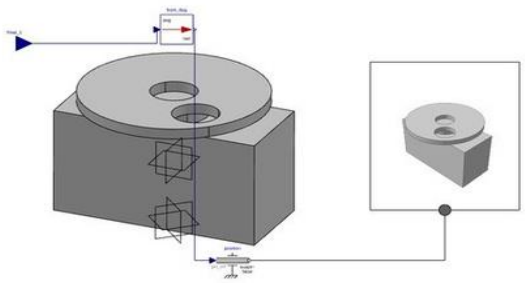
GPS Map Camera
Google
Vallam, Tamil Nadu, India
P3h9+xhm, Boys Hostel Rte, Vallam, Tamil Nadu 613403, India
Lat 10.729881° Long 79.06899°
Wednesday, 08/04/2026 03:58 PM GMT +05:30

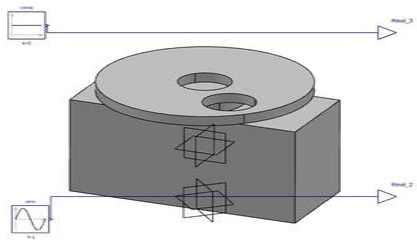
Task 4: RFLP

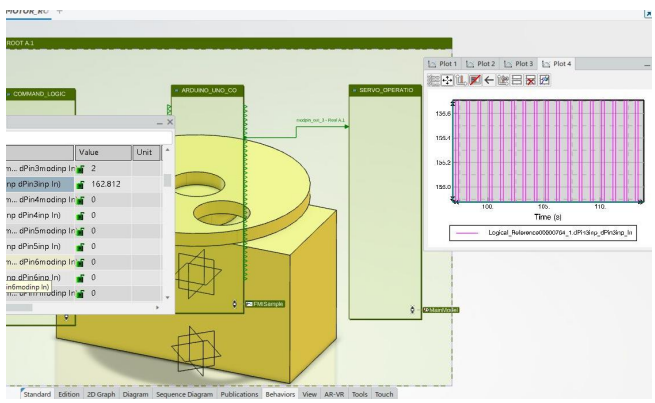


The servo motor experiment was structured through the RFLP methodology where requirements defined angular rotation accuracy and torque limits, functional blocks specified PWM signal generation and control logic, logical references mapped Arduino pin assignments such as Pin 9 for signal, 5V for power, and GND for ground, and physical models represented the CAD geometry of the servo housing, then Visual Studio was used to extend 3DEXPERIENCE behavior modeling with custom scripts, Arduino IDE compiled the control code using the Servo library with commands like servo.write(180);, the compiled code was linked to logical references in CATIA Functional & Logical Design, co-simulation was executed to synchronize mechanical rotation with electrical input, position sensors validated angular displacement in real time, plots displayed servo angle versus time to confirm smooth motion, logical constraints ensured pin-to-motion consistency across the digital twin, contact detection verified clearance during 180-degree rotation, digital twin integration allowed monitoring of variables such as input voltage and rotation angle, system behavior was analyzed under load conditions to check stability, RFLP traceability ensured that requirements were linked to functional, logical, and physical domains, Visual Studio integration supported communication between Arduino code and CATIA simulation, Arduino IDE provided compilation and debugging of embedded control logic, CATIA Behavior Modeling executed the servo's digital twin simulation, co-simulation confirmed mechanical feasibility and logical correctness, real-time graphs validated PWM input against angular output, logical references ensured synchronization between hardware pins and CAD motion, clearance analysis prevented collision during rotation, requirement validation confirmed servo performance against design specifications, functional mapping ensured PWM signals matched intended rotation angles, logical modeling connected Arduino inputs to servo

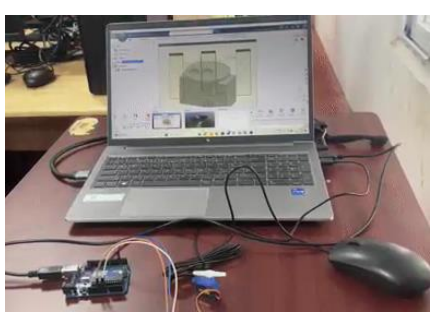
behavior, and physical modeling completed the digital twin by linking CAD geometry with simulation results.







Output:



Application:

Field	Application	Benefits
Manufacturing	Virtual replicas of machines and production lines.	Predictive maintenance, reduced downtime, optimized workflows.
Healthcare	Patient-specific digital twins (organs, treatments).	Personalized medicine, surgical planning, drug testing.
Automotive	Vehicle systems and autonomous driving simulations.	Safety validation, faster prototyping, real-time performance monitoring.
Aerospace	Aircraft engines and flight systems modeled virtually.	Reliability testing, predictive maintenance, reduced testing costs.

Benefits:

- **Cost Reduction:** Early detection of faults prevents expensive breakdowns.
- **Efficiency Gains:** Optimized processes improve productivity.
- **Innovation:** Faster prototyping and testing of new designs.
- **Sustainability:** Reduced waste and energy consumption.
- **Safety:** Risk-free testing in virtual environments.

Conclusion:

Digital twins are transforming industries by bridging the gap between physical and virtual systems. In your servo motor experiment, the digital twin allowed co-simulation of mechanical design and Arduino control logic. In broader fields, the same principle enables predictive maintenance in factories, personalized healthcare, smart city planning, and aerospace safety validation.

Source Link: https://github.com/nothingspacenot-source/VAC-DIGITAL-TWIN.git